

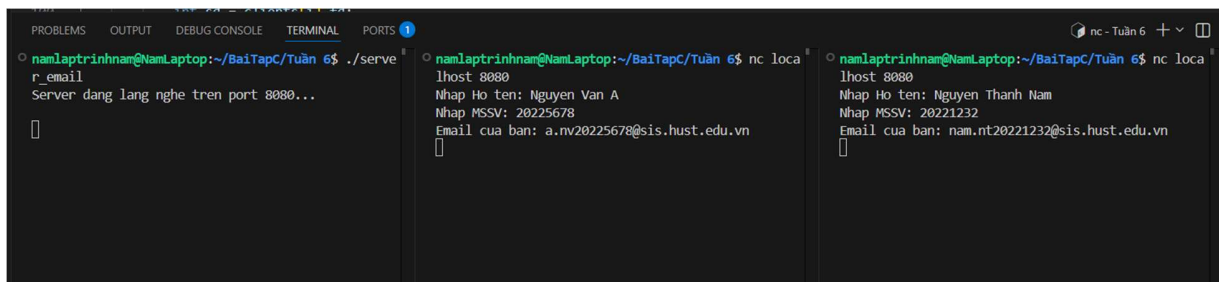
Báo cáo Bài tập áp dụng

Họ và tên: Nguyễn Thành Nam

MSSV: 20225368

Bài 1: Lập trình ứng dụng server trả về địa chỉ email của sinh viên theo yêu cầu sau:

- Khi có client kết nối, server gửi thông điệp để hỏi “Họ tên” và “MSSV”
- Từ dữ liệu nhận được, server phản hồi địa chỉ email cho client, theo cấu trúc email của sv ĐHBK
- Sử dụng non-blocking server, thử nghiệm với nhiều client kết nối cùng một lúc.



Hình 1. Minh họa server sử dụng kỹ thuật non-blocking phục vụ nhiều client kết nối đồng thời.

Giải thích kết quả:

- **Mô tả kịch bản:** Bức ảnh thể hiện 3 phiên làm việc trên terminal hoạt động song song.
 - **Cửa sổ bên trái:** Ứng dụng Server đã khởi động thành công và đang ở trạng thái lắng nghe (listen) các kết nối đến tại cổng 8080.
 - **Cửa sổ ở giữa (Client 1):** Một client kết nối thành công qua lệnh nc, tiến hành nhập thông tin họ tên "Nguyen Van A" và MSSV "20225678". Server xử lý chuỗi và trả về chính xác định dạng email ĐHBK là a.nv20225678@sis.hust.edu.vn.
 - **Cửa sổ bên phải (Client 2):** Đồng thời trong lúc đó, một client thứ hai cũng kết nối và nhập thông tin "Nguyen Thanh Nam", MSSV "20221232" và nhận được phản hồi email tương ứng là nam.nt20221232@sis.hust.edu.vn.

- **Đánh giá:** Kết quả chạy thử chứng minh tính đúng đắn của ứng dụng. Server đã áp dụng thành công cơ chế đa luồng/non-blocking (thông qua hàm `select()`) để quản lý nhiều file descriptor, cho phép tiếp nhận và xử lý yêu cầu từ Client 2 mà không bị treo hay phải chờ Client 1 ngắt kết nối. Thuật toán phân tích chuỗi họ tên và tạo email hoạt động chính xác theo chuẩn của trường.

Bài 2: Lập trình ứng dụng UDP chat sử dụng kỹ thuật non-blocking, cho phép vừa gửi và nhận dữ liệu trong một ứng dụng. Ứng dụng nhận 3 tham số dòng lệnh:

```
udp_chat port_s ip_d port_d
```

Trong đó `port_s` là cổng nhận dữ liệu trên ứng dụng hiện tại, `ip_d` và `port_d` là địa chỉ IP và cổng của ứng dụng cần kết nối

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 6$ gcc udp_chat.c -o udp_chat
namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 6$ ./udp_chat 8000 127.0.0.1 8001
UDP Chat khởi tạo tại cổng 8000.
Sẵn sàng gửi đến 127.0.0.1:8001

xin chao client 2, toi la client 1
>> Nhận:
>> Nhận: Xin chao client 1, toi la client 2
[]

namlaptrinhnam@NamLaptop:~/BaiTapC/Tuần 6$ ./udp_chat 8001 127.0.0.1 8000
UDP Chat khởi tạo tại cổng 8001.
Sẵn sàng gửi đến 127.0.0.1:8000

>> Nhận: xin chao client 2, toi la client 1
Xin chao client 1, toi la client 2
[]
  
```

Hình 2. Minh họa giao tiếp hai chiều giữa hai ứng dụng UDP Chat sử dụng cơ chế non-blocking.

Giải thích kết quả:

- **Mô tả kịch bản:** Bức ảnh hiển thị hai cửa sổ terminal đóng vai trò là hai người dùng đang chat với nhau thông qua giao thức UDP trên nền địa chỉ loopback (127.0.0.1).
 - **Cửa sổ bên trái (Client 1):** Khởi tạo ứng dụng lắng nghe tại cổng 8000 và cấu hình đích đến là cổng 8001. Client này chủ động gửi đi tin nhắn "*xin chao client 2, toi la client 1*", sau đó nhận được tin nhắn phản hồi hiển thị với tiền tố `>> Nhận:`.
 - **Cửa sổ bên phải (Client 2):** Khởi tạo ứng dụng lắng nghe tại cổng 8001 và cấu hình đích đến là cổng 8000. Cửa sổ này hiển thị tin nhắn đến từ Client 1 và thực hiện gửi trả lại thông điệp "*Xin chao client 1, toi la client 2*".
- **Đánh giá:** Giao diện cho thấy luồng nhập/xuất hoạt động trơn tru. Nhờ việc giám sát đồng thời luồng nhập từ bàn phím (`STDIN_FILENO`) và dữ liệu đến từ socket, chương trình không bị kẹt ở hàm đọc dữ liệu mạng (`recvfrom`) hay hàm đọc bàn phím (`fgets`). Cả hai client có thể vừa gõ tin nhắn vừa nhận tin nhắn bất đồng bộ, đáp ứng đúng yêu cầu của một ứng dụng chat thời gian thực.

